



NORTHERN UNIVERSITY OF BUSINESS & TECHNOLOGY KHULNA

Department of Computer Science and Engineering



SMART FARMER ASSISTANT

AI-Based Agriculture Support System



Submitted To:

Md. Riaz Mahmud

Assistant Professor, Department of Computer Science and Engineering

Northern University of Business and Technology Khulna

Submitted By:

Team Name: CSE4204-8D-T03

Team Leader: Sakib Foysal Ejarder

ID: 11220320948

Section: 8D

Submission date: 13th August 2026

CSE4204 — Mobile Computing Lab

Week 08 Submission

AI Integration Report

Smart Farmer Assistant

AI-Based Agriculture Support System

Table of Contents

Content	Page
1. Team Information	2
2. Why AI Is Required	3
3. Selected AI Platform and Model	3
4. AI Features Implemented	4
5. AI Workflow	7
6. Prompt Engineering Strategy and Examples	9
7. AI Response Handling and Output Validation	13
8. Limitations and Future Improvements	14
9. Secure Setup Notes	15

1. Team Information

Field	Details
Team Name	CSE4204-8D-T03
Section	8D
Course	CSE4204 — Software Engineering Lab
Semester	Spring 2026
Institution	Leading University / University (Bangladesh)
GitHub Repo	github.com/sakib-foysal/CSE4204-8D-T03-SmartFarmer-Assistant

1.1 Team Members

#	Name	Student ID	Role
1	Sakib Foysal Ejarder	11220320948	Team Leader / Backend
2	Md. Noyon Sheikh	11220320946	Frontend Developer
3	Sayed Tauhidul Islam	11220320950	AI / ML Engineer
4	Sayed Akib Osman	11220320974	Database / QA

Project Problem

Farmers often need quick, understandable guidance when a crop develops a symptom, when they want an initial opinion about a leaf image, or when they need to plan fertilizer use for a selected crop. A fixed FAQ or fixed recommendation card cannot account for the farmer's crop, symptom, language,

farm context, and follow-up questions. Incorrectly confident agricultural advice can also create safety and financial risks.

Smart Farmer Assistant addresses this gap with a bilingual (Bangla and English) web application. Its AI features provide a practical first response while clearly stating their limitations and directing farmers to local agricultural-extension guidance, product labels, soil tests, or an agriculture officer when appropriate.

2. Why AI Is Required

AI is required because agricultural questions are variable and context dependent. For example, a farmer may ask about a rice leaf symptom in Bangla, upload a tomato leaf image, or request a fertilizer planning guide for a specific crop. Maintaining a manually written answer for every combination would be impractical and would still be too rigid.

The application uses a generative AI model to:

1. Understand free-text questions in Bangla, English, or mixed language.
2. Provide contextual, short, farmer-friendly guidance.
3. Inspect visible clues in a crop/leaf image without claiming a confirmed diagnosis.
4. Generate a structured fertilizer-planning guide instead of displaying the same fixed card for every crop.

AI is not used as an autonomous decision maker. It supports the farmer with an initial, explainable response and preserves the role of qualified local agricultural professionals.

3. Selected AI Platform and Model

Item	Implementation
AI platform	Google Gemini API (Google AI Studio)
API style	Gemini generateContent REST endpoint over HTTPS
Configured model	gemini-3.5-flash by default; configurable through GEMINI_MODEL
Backend	Django REST Framework and Python requests

Item	Implementation
Frontend	React, TypeScript, Vite, Tailwind CSS
Database	PostgreSQL; successful chat, visual-assessment, and fertilizer-plan history is stored per authenticated user
Training approach	No custom model training; the system uses a pretrained Gemini model plus task-specific prompt engineering

The Gemini API key is read only by the Django backend from backend/.env as GEMINI_API_KEY. It is never sent to the React frontend and must not be committed to GitHub.

4. AI Features Implemented

4.1 Smart Farmer AI Chatbot

The chatbot accepts an authenticated farmer's question and returns concise agricultural guidance in the same language as the question. It supports topics such as crops, pests, diseases, irrigation, fertilizer, weather preparedness, and market planning.

Field	Detail
Frontend screen	/chatbot
Backend endpoint	POST /api/chat-history/ask/
Input	question (required, trimmed, maximum 1,200 characters)
Output	question, Gemini response, date, and model label
Persistence	Successful question-answer pairs are saved in ChatHistory and reloaded for that user

Example input: আমার ধানের পাতায় বাদামি দাগ হচ্ছে, কী করব?

Expected behavior: the chatbot gives a short, cautious initial checklist, asks one concise follow-up question if crop stage or location is needed, and reminds the farmer to follow local guidance and product labels before applying chemicals.

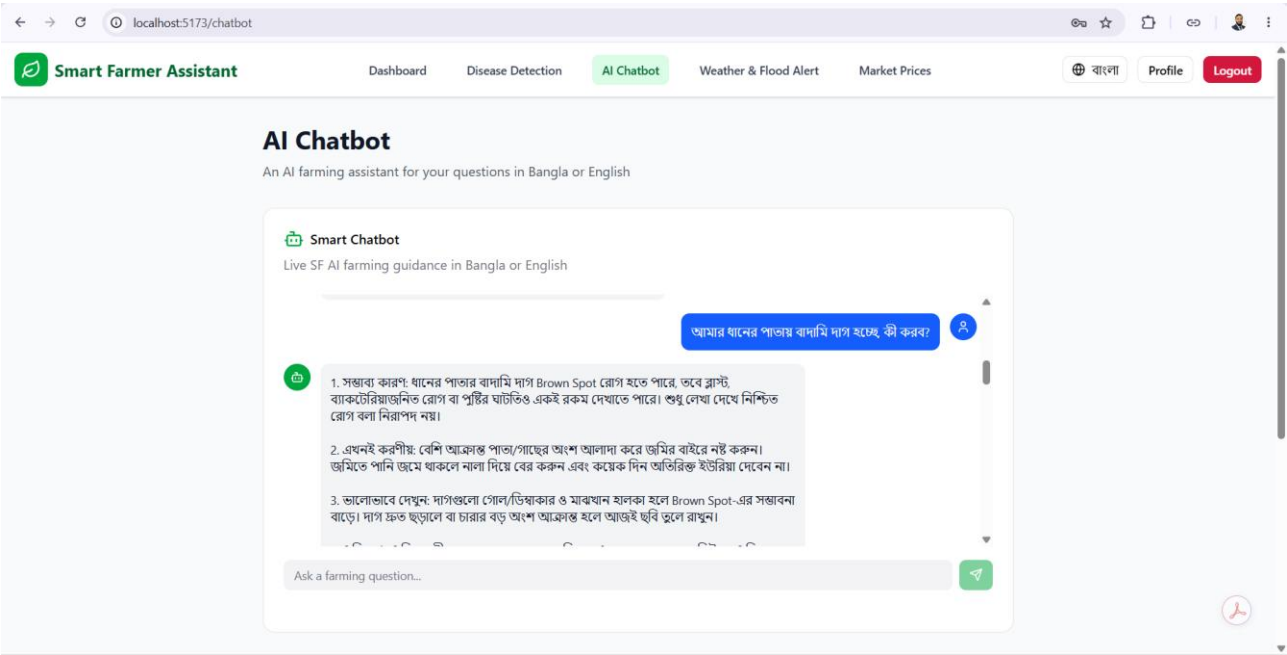


Figure 4.1: AI Chatbot Interface

4.2 Crop-Leaf Visual Assessment

The farmer uploads a PNG, JPG, or WEBP crop/leaf image. Gemini examines the visible content and returns a likely issue, visual confidence, safe next steps, and a disclaimer. This feature is intentionally described as a visual assessment, not a laboratory-confirmed diagnosis.

Field	Detail
Frontend screen	/disease-detection
Backend endpoint	POST /api/disease-history/analyze/
Input	Base64 image data, optional crop hint
Accepted file types	PNG, JPG/JPEG, WEBP
Maximum size	2 MB
Output	Prediction, 0-100 visual confidence, treatment/next steps, disclaimer

Field	Detail
Persistence	Successful results are stored in DiseaseHistory

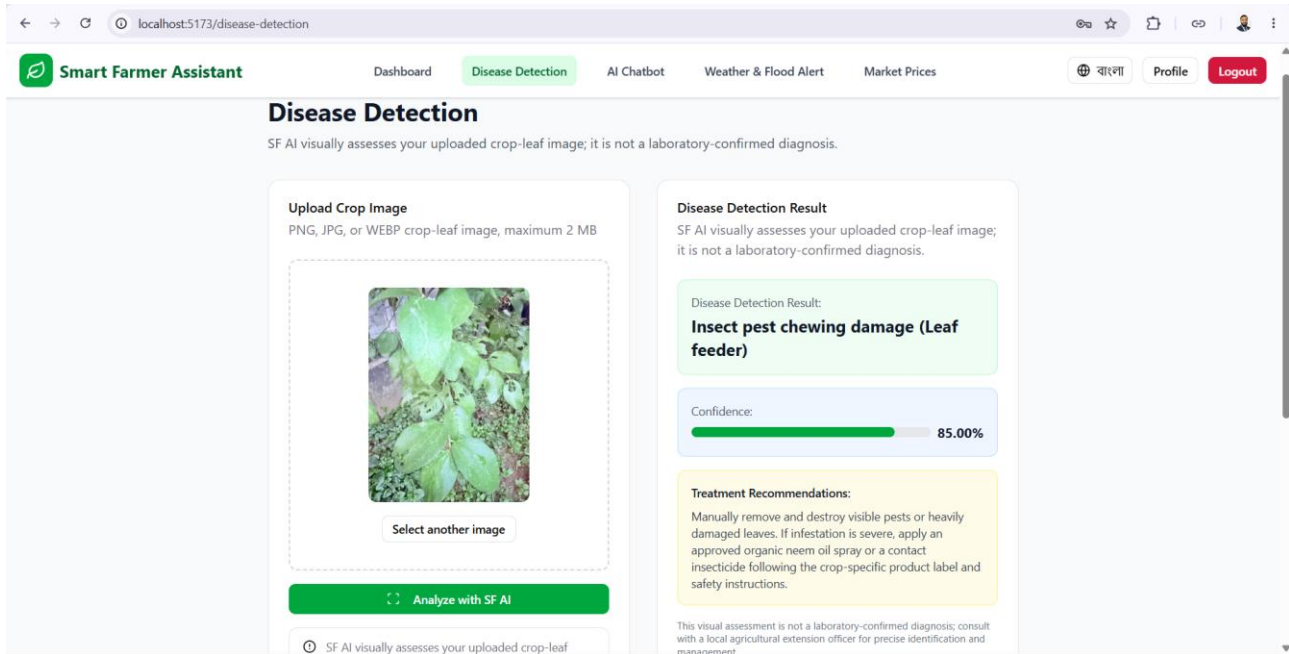


Figure 4.2: Disease Detection Interface

4.3 AI-Generated Fertilizer Planning Guide

The farmer selects a crop and may provide brief farm context. Gemini returns a crop-specific planning guide containing fertilizer items, general amount guidance, timing, tips, and a safety disclaimer. The prompt prevents the output from being presented as a universal fertilizer prescription.

Field	Detail
Frontend screen	/fertilizer
Backend endpoint	POST /api/fertilizer-recommendations/generate/
Input	Crop name (required, max 100 characters), optional farm context (max 500 characters)
Output	Structured JSON with crop, fertilizers, tips, and disclaimer
Persistence	Generated plan summary is stored in FertilizerRecommendation

Fertilizer Recommendation

AI-generated guide; not a universal fertilizer prescription

Selected crop

Crop shown in the uploaded image

Balanced fertilizer plan

Use a balanced NPK fertilizer only according to a soil test and the product label.

Apply in split doses during land preparation and the crop's growth stages.

Organic matter

Use well-decomposed compost or manure according to local guidance.

Incorporate it well into the soil during land preparation.

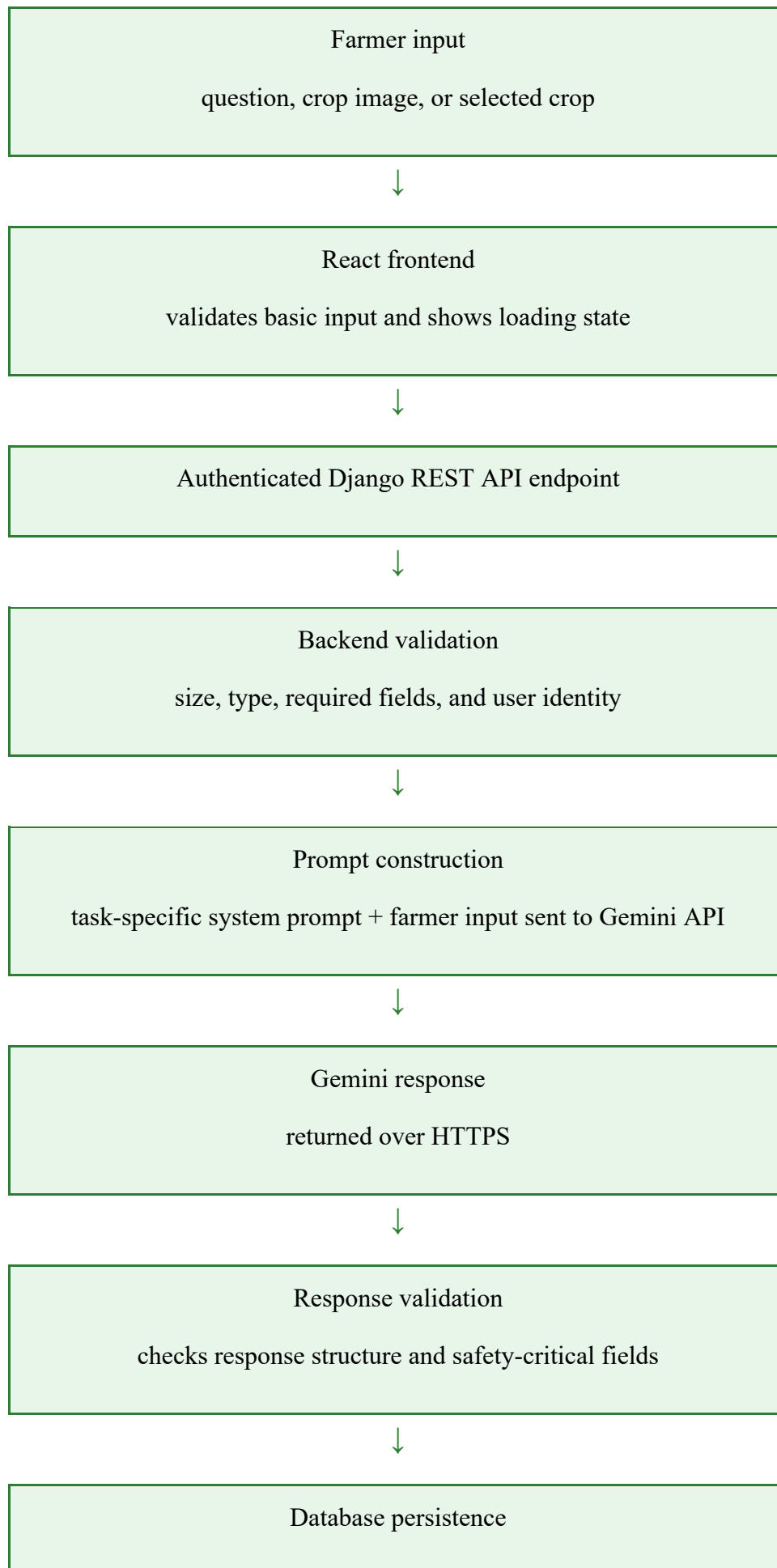
- ✓ Do not apply too much fertilizer at once; split applications according to crop growth.
- ✓ Irrigate as needed after fertilizer application, without allowing standing water.
- ✓ Avoid excessive nitrogen while the crop has active pest or disease damage.

 This is a safe general guide. Confirm the exact dose with a soil test, crop stage, and product label.

Figure 4.3: Fertilizer Planning

5. AI Workflow

The diagram below shows how a farmer's input travels through the application, is processed by Gemini, and is returned as a validated, safely-displayed result.



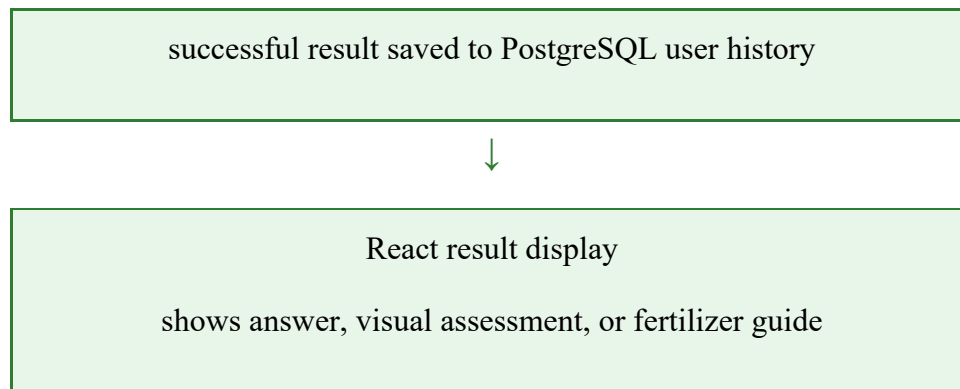


Figure 1. End-to-end AI workflow, from farmer input through Gemini processing to the displayed result.

Step-by-Step Explanation

1. Farmer input: The user writes a question, uploads a crop image, or selects a crop.
2. Frontend: React trims text, restricts the chatbot input length, prevents duplicate submit while a request is in progress, and shows a loading state.
3. Authentication: The frontend sends the user's JWT token in the Authorization: Bearer <token> header.
4. Backend validation: Django REST Framework validates the input before calling AI. The image endpoint checks its data URL, MIME type, base64 validity, and 2 MB size limit.
5. Prompt construction: The backend selects a purpose-built prompt for chat, image assessment, or fertilizer planning, then adds the validated user input.
6. AI service call: The backend calls Gemini's generateContent endpoint. The API key remains on the server.
7. Response validation: Django rejects empty, malformed, or wrongly structured responses. The fertilizer response must contain valid fertilizers and tips arrays; image confidence is normalized to 0-100.
8. Database persistence: Only successful results are recorded under the authenticated user's history.
9. Frontend display: React displays the result with clear labels and a safe error alert when the service cannot answer.

6. Prompt Engineering Strategy and Examples

Strategy

The prompts are designed to make the model helpful without overstating certainty. They set the model's role, Bangladesh context, language behavior, answer length, mandatory safety limitations, and output format. The backend uses separate prompts for text advice, visual assessment, and structured fertilizer output because each task has different validation needs.

A. Chatbot Prompt

System prompt used by the backend

You are Smart Farmer Assistant, a careful agriculture support assistant for farmers in Bangladesh.

Answer in the same language as the farmer (Bangla, English, or mixed Bangla-English). Give practical, short, numbered advice about crops, pests, diseases, irrigation, fertilizer, weather preparedness, and market planning.

Ask one concise follow-up question when crop, symptom, location, growth stage, or soil condition is needed. Clearly state that exact pesticide/fertilizer dose must follow local agricultural-extension guidance and product labels. Do not diagnose with certainty from text alone, do not invent weather or market data, and recommend a local agriculture officer for severe crop loss, poisoning, or urgent safety issues.

Do not use Markdown tables. Keep the response under 220 words.

User prompt example

আমার ধানের পাতায় বাদামি দাগ হচ্ছে, কী করব?

Expected output

1. সম্ভাব্য কারণ: ধানের পাতার বাদামি দাগ Brown Spot রোগ হতে পারে, তবে ব্লাস্ট, ব্যাকটেরিয়াজনিত রোগ বা পুষ্টির ঘাটতিও একই রকম দেখাতে পারে। শুধু লেখা দেখে নিশ্চিত রোগ বলা নিরাপদ নয়।

2. এখনই করণীয়: বেশি আক্রান্ত পাতা/গাছের অংশ আলাদা করে জমির বাইরে নষ্ট করুন। জমিতে পানি জমে থাকলে নালা দিয়ে বের করুন এবং কয়েক দিন অতিরিক্ত ইউরিয়া দেবেন না।
3. ভালোভাবে দেখুন: দাগগুলো গোল/ডিম্বাকার ও মাঝখান হালকা হলে Brown Spot-এর সম্ভাবনা বাড়ে। দাগ দ্রুত ছড়ালে বা চারার বড় অংশ আক্রান্ত হলে আজই ছবি তুলে রাখুন।
4. প্রতিরোধ: পরিষ্কার বীজ ব্যবহার, সুষম সার, জমির আগাছা ও আক্রান্ত অবশিষ্টাংশ পরিষ্কার রাখা এবং গাছের বাতাস চলাচল নিশ্চিত করুন।
5. ওষুধ: নিজে থেকে ডোজ ঠিক করবেন না। স্থানীয় কৃষি অফিস/উপসহকারী কৃষি কর্মকর্তা দেখে অনুমোদিত ছত্রাকনাশক ও লেবেল অনুযায়ী প্রয়োগের পরামর্শ দিন।
6. প্রশ্ন: ধান এখন চারা, কুশি নাকি শীষ বের হওয়ার পর্যায়ে? সম্ভব হলে পাতার একটি পরিষ্কার ছবি দিন।

B. Crop-Leaf Visual-Assessment Prompt

System prompt used by the backend

You are an agricultural visual-assessment assistant for Bangladesh. Analyze only what is visibly supported by this crop/leaf image. Do not claim a laboratory-confirmed diagnosis. If this is not a crop/leaf image, say so.

Reply in plain text using these exact labels on separate lines:

PREDICTION: short likely issue or 'Disease visible';

CONFIDENCE: a visual confidence number from 0 to 100;

TREATMENT: short safe next steps; DISCLAIMER: one concise safety note.

Use Bangla when the crop hint is Bangla, otherwise English. Avoid exact chemical dose; advise confirming pesticide/fungicide choices with local agricultural extension and product labels.

User prompt example

Assess this image.

Crop hint supplied by farmer: Put any image(Like Puisak)

Expected output

AI visual analysis is temporarily unavailable. Upload a clear close image of both sides of a leaf in good light. Isolate badly affected leaves, keep the field clean, and avoid standing water.

C. Fertilizer-Plan Prompt

System prompt used by the backend

You are a careful Bangladesh agriculture assistant. Create a general fertilizer planning guide, not a prescription. Return JSON only with keys: crop, fertilizers (array of objects with name, amount, timing), tips (array of strings), disclaimer. Never invent soil-test results and never present exact application rates as universally correct; clearly ask the farmer to confirm dose with a soil test, local agricultural extension officer, and product label. Use simple language.

User prompt example

Crop: No prompt needed. AI detects disease from the image and recommend fertilizer.

Expected output

Organic matter

Use well-decomposed compost or manure according to local guidance.

Incorporate it well into the soil during land preparation.

- ✔ Do not apply too much fertilizer at once; split applications according to crop growth.
- ✔ Irrigate as needed after fertilizer application, without allowing standing water.
- ✔ Avoid excessive nitrogen while the crop has active pest or disease damage.

⚠ This is a safe general guide. Confirm the exact dose with a soil test, crop stage, and product label.

Figure 6.1: Recommended Fertilizer

7. AI Response Handling and Output Validation

Scenario	Backend Handling	Frontend Behavior
Valid response	Parses and validates the result; saves it in user history	Displays the dynamic AI result
Empty question	Rejects before AI call	User cannot submit an empty chat message
Oversized question	Rejects above 1,200 characters	Input is limited to 1,200 characters
Invalid image	Rejects non-image, unsupported type, invalid base64, or over-2-MB image	Shows API error; does not display a false result
Empty Gemini response	Returns a clear retry message	Shows an alert message

Scenario	Backend Handling	Frontend Behavior
Invalid JSON / malformed structured result	Rejects it with HTTP 502	Shows an alert instead of malformed data
Network / API service failure	Returns safe HTTP 503 message	App remains usable; farmer may retry
API timeout	Uses a 25-second timeout and returns HTTP 504	Shows a retry message
Rate limit	Returns HTTP 429 with wait-and-retry message	Shows the message without crashing
Invalid/missing API key	Returns configuration-safe error; key is not exposed	Shows a contact-administrator message
Uncertain diagnosis or dosage	Prompt requires qualification and disclaimer	Renders disclaimer with result

The HTTP client retries only transient failures, uses TLS, and does not bypass certificate validation. The API key remains server-side in backend/.env.

8. Limitations and Future Improvements

Current Limitations

- AI advice is an initial support response, not a replacement for field inspection, laboratory testing, or an agricultural officer.
- The model does not currently receive verified real-time weather, market-price, soil-test, or precise farm-location information.
- Crop-image analysis is visual only; image quality, lighting, leaf angle, and missing field context can affect reliability.
- The fertilizer feature is a planning guide, not a universal dose prescription.
- AI availability depends on network connectivity, provider service status, configured API quota, and a valid server-side API key.

Future Improvements

1. Add consent-based farm profiles with crops, district, soil-test, and growth-stage context.
2. Add retrieval-augmented generation (RAG) from verified Bangladesh agricultural-extension documents.
3. Integrate live weather and market data as clearly labelled context.
4. Add Bangla voice input/output for improved accessibility.
5. Collect farmer feedback on response usefulness and perform safety evaluation.
6. Build a separately validated crop-disease classifier using licensed labelled data and agricultural-expert review.
7. Add observability, quota monitoring, and an administrator dashboard for AI-service health.

9. Secure Setup Notes

Create backend/.env locally and add the key without sharing or committing it:

```
GEMINI_API_KEY=G=gemini_private_key_here  
GEMINI_MODEL=gemini-3.5-flash  
GEMINI_TIMEOUT_SECONDS=25
```

The secret must remain in backend/.env, which should be included in .gitignore. It must never be placed in frontend source code, screenshots, documentation examples, or GitHub commits.